

10. НЕБЕЗОПАСНЫЙ КОД (UNSAFE CODE)

Само упоминание небезопасного кода (`unsafe code`) часто вызывает сильную реакцию у многих в сообществе Rust, а также у многих из тех, кто наблюдает за Rust со стороны. В то время как одни утверждают, что это «не такая уж большая проблема», другие осуждают его как «причину, по которой все обещания Rust — ложь». В этой главе я надеюсь немного приоткрыть завесу и объяснить, что такое `unsafe`, чем он не является и как следует подходить к его безопасному использованию. На момент написания книги — и, вероятно, всё ещё на момент, когда вы это читаете, — точные правила, определяющие требования Rust к небезопасному коду, всё ещё прорабатываются, и даже если бы они все были окончательно определены, их полное описание выходило бы за рамки этой книги. Вместо этого я постараюсь вооружить вас строительными блоками, интуицией и инструментарием, которые понадобятся вам, чтобы проложить себе путь через большую часть небезопасного кода.

Главный вывод, который вы должны сделать из этой главы, таков: небезопасный код — это механизм, который Rust предоставляет разработчикам для того, чтобы воспользоваться инвариантами (`invariants`), которые компилятор по той или иной причине проверить не может. Мы рассмотрим способы, которыми `unsafe` это делает, что это могут быть за инварианты и что мы можем с ними сделать в результате.

ИНВАРИАНТЫ (INVARIANTS) На протяжении этой главы я буду много говорить об инвариантах. *Инвариант* — это просто причудливый способ сказать «нечто, что должно быть истинным, чтобы ваша программа работала правильно». Например, в Rust один из инвариантов состоит в том, что ссылки, использующие `&` и `&mut`, никогда не должны «висеть» (`dangle`) — они всегда указывают на корректные значения. У вас также могут быть инварианты, специфичные для приложения или библиотеки, такие как «головной указатель всегда впереди хвостового указателя» или «ёмкость всегда является степенью двойки». В конечном счёте инварианты представляют собой все предположения, необходимые для того, чтобы

ваш код был корректен. Однако вы не всегда можете осознавать все инварианты, которые использует ваш код, и именно здесь закрадываются ошибки.

Принципиально важно, что небезопасный код — это не способ обойти различные правила Rust, такие как проверка заимствований (borrow checking), а скорее способ обеспечить соблюдение этих правил, используя рассуждения, недоступные компилятору. Когда вы пишете небезопасный код, на вас как на автора ложится обязанность гарантировать, что получившийся код безопасен. В каком-то смысле `unsafe` как ключевое слово вводит в заблуждение, когда оно используется для разрешения небезопасных операций через `unsafe {}`: дело не в том, что содержащийся код *является* небезопасным, а в том, что коду разрешено выполнять операции, которые в иных обстоятельствах были бы небезопасны, потому что в этом конкретном контексте эти операции безопасны.

Оставшаяся часть этой главы разделена на четыре раздела. Мы начнём с краткого рассмотрения того, как используется само ключевое слово, затем изучим, что `unsafe` позволяет вам делать. Далее мы рассмотрим, что вы обязаны обеспечить, чтобы писать безопасный небезопасный код. Наконец, я дам несколько советов о том, как на практике подходить к безопасному написанию небезопасного кода.

Ключевое слово `unsafe` (The unsafe Keyword)

Прежде чем мы обсудим возможности, которые предоставляет `unsafe`, нам нужно поговорить о двух его разных значениях. Ключевое слово `unsafe` в Rust выполняет двойную функцию: оно помечает определённую функцию как небезопасную для вызова *и* позволяет вам вызывать небезопасную функциональность в определённом блоке кода. Например, метод в листинге 10-1 помечен как небезопасный, хотя он не содержит небезопасного кода. Здесь ключевое слово `unsafe` служит предупреждением для вызывающей стороны о том, что существуют дополнительные гарантии, которые должен вручную проверить тот, кто пишет код, вызывающий `decr`.

```
impl<T> SomeType<T> {  
    pub unsafe fn decr(&self) {  
        self.some_size -= 1;  
    }  
}
```

```
}  
}
```

Листинг 10-1: Небезопасный метод, содержащий только безопасный код

Листинг 10-2 иллюстрирует второе использование. Здесь сам метод не помечен как небезопасный, хотя он содержит небезопасный код.

```
impl<T> SomeType<T> {  
    pub fn as_ref(&self) -> &T {  
        unsafe { &*self.ptr }  
    }  
}
```

Листинг 10-2: Безопасный метод, содержащий небезопасный код

Эти два листинга различаются в использовании `unsafe`, потому что они воплощают разные контракты. `decr` требует, чтобы вызывающая сторона была осторожна при вызове этого метода, тогда как `as_ref` предполагает, что вызывающая сторона *была* осторожна при вызове других небезопасных методов (таких как `decr`). Чтобы понять почему, представьте, что `SomeType` на самом деле является типом с подсчётом ссылок, таким как `Rc`. Хотя `decr` всего лишь уменьшает число, это уменьшение может в свою очередь спровоцировать неопределённое поведение через безопасный метод `as_ref`. Если вы вызовете `decr`, а затем сбросите (`drop`) предпоследний `Rc` для данного `T`, счётчик ссылок упадёт до нуля, и `T` будет сброшен — но программа всё ещё может вызвать `as_ref` для последнего `Rc` и в итоге получить висячую (`dangling`) ссылку.

Неопределённое поведение (undefined behavior) описывает последствия программы, которая нарушает инварианты языка во время выполнения. В общем случае, если программа провоцирует неопределённое поведение, результат становится совершенно непредсказуемым. Мы рассмотрим неопределённое поведение более подробно далее в этой главе.

И наоборот, поскольку нет способа повредить счётчик ссылок `Rc` с помощью безопасного кода, всегда безопасно разыменовывать указатель внутри `Rc` так, как это делает код `as_ref`: тот факт, что `&self` существует, является доказательством того, что указатель всё ещё должен быть корректен. Мы можем использовать это, чтобы предоставить вызывающей стороне

безопасный API для операции, которая в иных обстоятельствах была бы небезопасной, — что является основной частью того, как ответственно использовать `unsafe`.

По историческим причинам каждая функция `unsafe fn` сегодня в Rust содержит неявный небезопасный блок. То есть, если вы объявляете `unsafe fn`, вы всегда можете вызывать любые небезопасные методы или примитивные операции внутри этой функции. Однако теперь это решение считается ошибкой, и в настоящее время оно отменяется через принятый и реализованный RFC 2585. Этот RFC делает предупреждением наличие `unsafe fn`, которая выполняет небезопасные операции без явного небезопасного блока внутри неё. Этот линт, вероятно, также станет жёсткой ошибкой в будущих изданиях (editions) Rust. Идея состоит в том, чтобы уменьшить «радиус поражения от выстрела себе в ногу» (foot-gun radius): если каждая `unsafe fn` является одним гигантским небезопасным блоком, то вы можете случайно выполнить небезопасные операции, даже не осознавая этого! Например, в `decr` из листинга 10-1, согласно текущим правилам, вы также могли бы добавить `*std::ptr::null()` без какой-либо аннотации `unsafe`.

Различие между `unsafe` как маркером и небезопасными блоками как механизмом для включения небезопасных операций важно, потому что вы должны думать о них по-разному. `unsafe fn` указывает вызывающей стороне, что она должна быть осторожна при вызове рассматриваемой функции и что она должна обеспечить соблюдение задокументированных инвариантов безопасности этой функции.

Между тем небезопасный блок представляет собой утверждение о том, что тот, кто написал этот блок, тщательно проверил, что инварианты безопасности для любых небезопасных операций, выполняемых внутри него, соблюдаются. Если хотите приблизительную аналогию из реального мира, `unsafe fn` — это незаполненный контракт, который просит автора вызывающего кода «торжественно поклясться в X, Y и Z». Между тем `unsafe {}` — это подпись автора вызывающего кода под всеми небезопасными контрактами, содержащимися в блоке. Держите это в уме, пока мы будем разбирать остальную часть этой главы.

Большая сила (Great Power)

Итак, как только вы подписываете небезопасный контракт через `unsafe {}`, что вам разрешается делать? Честно говоря, не так уж много. Точнее, это не включает никаких новых возможностей. Внутри небезопасного блока вам разрешается разыменовывать сырые указатели (raw pointers) и вызывать `unsafe fn`.

И всё. Технически есть ещё несколько вещей, которые вы можете делать, например, обращаться к изменяемым и внешним статическим переменным и обращаться к полям объединений (unions), но они на самом деле не сильно меняют обсуждение. И, честно говоря, этого достаточно. В совокупности эти возможности позволяют вам сеять всевозможный хаос, например, превращать типы друг в друга с помощью `mem::transmute`, разыменовывать указатели, указывающие кто знает куда, приводить `&a` к `&'static` или делать типы разделяемыми между границами потоков, даже если они не потокобезопасны.

В этом разделе мы не будем слишком беспокоиться о том, что может пойти не так с этими возможностями. Это мы оставим для скучного, ответственного, взрослого раздела, который последует за этим. Вместо этого мы рассмотрим эти изящные блестящие новые игрушки и то, что мы можем с ними делать.

Жонглирование сырыми указателями (Juggling Raw Pointers)

Одна из самых фундаментальных причин использовать `unsafe` — работать с типами сырых указателей Rust: `*const T` и `*mut T`. Вы должны думать о них как о более или менее аналогах `&T` и `&mut T`, за исключением того, что они не имеют времён жизни (lifetimes) и не подчиняются тем же правилам корректности, что и их аналоги с `&`, которые мы обсудим далее в этой главе. На эти типы взаимозаменяемо ссылаются как на *указатели (pointers)* и *сырые указатели (raw pointers)*, в основном потому, что многие разработчики инстинктивно называют ссылки указателями, и название их сырыми указателями делает различие более ясным.

Поскольку к `*` применяется меньше правил, чем к `&`, вы можете приводить ссылку к указателю даже вне небезопасного блока. Только если вы хотите пойти в обратную сторону, от `*` к `&`, вам нужен `unsafe`. Обычно вы превращаете указатель обратно в ссылку, чтобы делать полезные вещи с данными, на которые он указывает, такие как чтение или изменение их значения. По этой

причине распространённая операция с указателями — `unsafe { &*ptr }` (или `&mut *`). `*` здесь может выглядеть странно, поскольку код всего лишь конструирует ссылку, а не разыменовывает указатель, но это обретает смысл, если посмотреть на типы: если у вас есть `*mut T` и вы хотите `&mut T`, то `&mut ptr` дал бы вам просто `&mut *mut T`. Вам нужен `*`, чтобы указать, что вы хотите изменяемую ссылку на то, *на что* указывает `ptr`.

ТИПЫ УКАЗАТЕЛЕЙ (POINTER TYPES) Возможно, вам интересно, в чём разница между `*mut T`, `*const T` и `std::ptr::NonNull<T>`. Что ж, точная спецификация всё ещё прорабатывается, но основное практическое различие между `*mut T` и `*const T/NonNull<T>` состоит в том, что `*mut T` инвариантен (invariant) по `T` (см. раздел «Вариантность времён жизни» (Lifetime Variance) на странице XX в главе 2), тогда как два других ковариантны (covariant). Как следует из названий, `*const T` и `NonNull<T>` различаются в основном тем, что `NonNull<T>` не разрешено быть нулевым (null) указателем, тогда как `*const T` — разрешено.

Мой лучший совет при выборе между этими типами — использовать вашу интуицию о том, написали бы вы `&mut` или `&`, если бы могли назвать соответствующее время жизни. Если вы написали бы `&` и знаете, что указатель никогда не нулевой, используйте `NonNull<T>`. Он выигрывает от классной оптимизации, называемой *нишевой оптимизацией* (*niche optimization*): по сути, поскольку компилятор знает, что тип никогда не может быть нулевым, он может использовать эту информацию для представления типов вроде `Option<NonNull<T>>` без каких-либо дополнительных накладных расходов, так как вариант `None` можно представить, установив `NonNull` в нулевой указатель! Значение нулевого указателя — это ниша в типе `NonNull<T>`. Если указатель может быть нулевым, используйте `*const T`. А если бы вы написали `&mut T`, используйте `*mut T`.

Непредставимые времена жизни (Unrepresentable Lifetimes)

Поскольку сырые указатели не имеют времён жизни, их можно использовать в обстоятельствах, где живучесть значения, на которое указывают, нельзя выразить статически в системе времён жизни Rust, например, как самоуказатель в самоссылающейся структуре, подобной генераторам из главы 9. Указатель, который указывает внутрь `self`, корректен до тех пор, пока `self`

существует (и не перемещается, для чего и нужен `Pin`), но это не время жизни, которое вы можете в общем случае назвать. И хотя весь самоссылающийся тип может быть `'static`, самоуказатель — нет: если бы он был статическим, это подразумевало бы, что даже если бы вы отдали этот указатель кому-то другому, он мог бы продолжать использовать его вечно, даже после того, как `self` исчез! Возьмём в качестве примера тип в листинге 10-3; здесь мы пытаемся хранить сырые байты, составляющие значение, рядом с его сохранённым представлением.

```
struct Person<'a> {
    name: &'a str,
    age: usize,
}
struct Parsed {
    bytes: [u8; 1024],
    parsed: Person<'??>,
}
```

Листинг 10-3: Попытка (неудачная) назвать время жизни самоссылающейся ССЫЛКИ

Ссылка внутри `Person` хочет ссылаться на данные, хранящиеся в `bytes` в `Parsed`, но не существует времени жизни, которое мы могли бы присвоить этой ссылке из `Parsed`. Это не `'static` и не что-то вроде `'self` (которого не существует), потому что если `Parsed` перемещается, ссылка перестает быть корректной.

Поскольку указатели не имеют времён жизни, они обходят эту проблему, так как вам не нужно называть время жизни. Вместо этого вам просто нужно убедиться, что когда вы используете указатель, он всё ещё корректен, что вы и подтверждаете своей подписью, когда пишете `unsafe { &*ptr }`. В примере из листинга 10-3 `Person` вместо этого хранил бы `*const str`, а затем небезопасно превращал бы его в `&str` в те моменты, когда может гарантировать, что указатель всё ещё корректен.

Похожая проблема возникает с типом вроде `Arc`, который имеет указатель на значение, разделяемое в течение некоторой длительности, но эта длительность известна только во время выполнения, когда сбрасывается последний `Arc`. Указатель вроде как, в некотором роде, `'static`, но не совсем — как и в самоссылающемся случае, указатель перестает быть корректным, когда исчезает последняя ссылка `Arc`. Так что время жизни больше похоже на `'self`. У родственника `Arc` — `Weak` — время жизни также «пока не исчезнет

последний Arc», но поскольку Weak не является Arc, время жизни даже не привязано к self. Так что Arc и Weak оба используют сырые указатели внутри.

Арифметика указателей (Pointer Arithmetic)

С сырыми указателями вы можете выполнять произвольную арифметику указателей, прямо как в C, используя `.offset()`, `.add()` и `.sub()`, чтобы переместить указатель к любому байту, который находится внутри той же аллокации. Это чаще всего используется в высокооптимизированных по памяти структурах данных, таких как хеш-таблицы, где хранение дополнительного указателя для каждого элемента добавляло бы слишком много накладных расходов, а использование срезов (slices) невозможно. Это довольно нишевые случаи использования, и мы не будем больше говорить о них в этой книге, но я призываю вас прочитать код `hashbrown::RawTable` (<https://github.com/rust-lang/hashbrown/>), если хотите узнать больше!

Методы арифметики указателей небезопасны для вызова, даже если впоследствии вы не собираетесь превращать указатель в ссылку. Тому есть несколько причин, но главная состоит в том, что недопустимо заставлять указатель указывать за пределы аллокации, на которую он изначально указывал. Это провоцирует неопределённое поведение, и компилятору разрешено решить съесть ваш код и заменить его произвольной бессмыслицей, которую мог бы понять только компилятор. Если вы всё же используете эти методы, читайте документацию внимательно!

К указателю и обратно (To Pointer and Back Again)

Часто, когда вам нужно использовать указатели, это потому, что у вас есть обычный тип Rust, такой как ссылка, срез или строка, и вам нужно ненадолго переместиться в мир указателей, а затем вернуться к исходному обычному типу. Поэтому некоторые из ключевых типов стандартной библиотеки предоставляют вам способ превратить их в составные сырые части, такие как указатель и длина для среза, и способ превратить их обратно в целое, используя те же самые части. Например, вы можете получить указатель на данные среза с помощью `as_ptr` и его длину с помощью `[].len`. Затем вы можете восстановить срез, передав эти же значения в `std::slice::from_raw_parts`. У `Vec`, `Arc` и `String` есть похожие методы, которые возвращают сырой указатель на лежащую в основе аллокацию, а у `Box` есть

`Box::into_raw` и `Box::from_raw`, которые делают то же самое.

Вольное обращение с типами (Playing Fast and Loose with Types)

Иногда у вас есть тип `T`, и вы хотите обращаться с ним как с каким-то другим типом `U`. Будь то потому, что вам нужен молниеносный разбор без копирования (zero-copy parsing), или потому, что вам нужно повозиться с какими-то временами жизни, Rust предоставляет вам некоторые (очень небезопасные) инструменты для этого.

Первый и, безусловно, наиболее широко используемый из них — приведение указателей (pointer casting): вы можете привести `*const T` к любому другому `*const U` (и так же для `mut`), и вам даже не нужен для этого `unsafe`. Небезопасность вступает в игру только тогда, когда вы позже пытаетесь использовать приведённый указатель как ссылку, поскольку вам приходится утверждать, что сырой указатель действительно можно использовать как ссылку на тип, на который он указывает.

Такого рода приведение типов указателей особенно полезно при работе с интерфейсами внешних функций (foreign function interfaces, FFI) — вы можете привести любой указатель Rust к `*const std::ffi::c_void` или `*mut std::ffi::c_void`, а затем передать его C-функции, ожидающей указатель `void`. Аналогично, если вы получаете указатель `void` из C, который вы ранее передали туда, вы можете тривиально привести его обратно к исходному типу.

Приведения указателей также полезны, когда вы хотите интерпретировать последовательность байтов как обычные старые данные — типы вроде целых чисел, булевых значений, символов и массивов или структур `#[repr(C)]` из них — или записать такие типы напрямую как поток байтов без сериализации. При этом нужно держать в уме много инвариантов безопасности, но это мы оставим на потом.

Вызов небезопасных функций (Calling Unsafe Functions)

Можно утверждать, что наиболее часто используемая возможность `unsafe` состоит в том, что он позволяет вам вызывать небезопасные функции. На более глубоком уровне большинство этих функций небезопасны, потому что они оперируют сырыми указателями на каком-то фундаментальном уровне, но выше по стеку вы, как правило, взаимодействуете с небезопасностью прежде

всего через вызовы функций.

На самом деле нет предела тому, что может позволить вызов небезопасной функции, поскольку всё зависит от того, с какими библиотеками вы в итоге взаимодействуете. Но *в общем случае* небезопасные функции можно разделить на три лагеря: те, которые взаимодействуют с не-Rust-интерфейсами, те, которые пропускают проверки безопасности, и те, у которых есть пользовательские инварианты.

Интерфейсы внешних функций (Foreign Function Interfaces)

Rust позволяет вам объявлять функции и статические переменные, определённые на языке, отличном от Rust, с помощью блоков `extern` (которые мы подробно обсудим в главе 12). Когда вы объявляете такой блок, вы говорите Rust, что реализация элементов, появляющихся внутри него, будет предоставлена каким-то внешним источником при компоновке (linking) финального бинарного файла программы, например, C-библиотекой, которую вы интегрируете. Поскольку `extern`-элементы существуют вне контроля Rust, они по своей природе небезопасны для доступа. Если вы вызываете C-функцию из Rust, все гарантии отменяются — она может перезаписать всё содержимое вашей памяти и затереть все ваши аккуратно расставленные ссылки случайными указателями куда-то в ядро. Аналогично, `extern`-статическая переменная может быть изменена внешним кодом в любой момент и может быть заполнена всевозможными плохими байтами, которые не отражают её объявленный тип. Однако внутри небезопасного блока вы можете обращаться к `extern`-элементам сколько душе угодно, при условии, что вы готовы поручиться за то, что другая сторона `extern` ведёт себя в соответствии с правилами Rust.

Обойдусь без проверок безопасности (I'll Pass on Safety Checks)

Некоторые небезопасные операции можно сделать полностью безопасными, введя дополнительные проверки во время выполнения. Например, доступ к элементу в срезе небезопасен, поскольку вы можете попытаться обратиться к элементу за пределами длины среза. Но, учитывая, насколько распространена эта операция, было бы досадно, если бы индексация в срез была небезопасной. Вместо этого безопасная реализация включает проверки границ, которые (в зависимости от используемого метода) либо паникуют, либо возвращают

Option, если индекс выходит за границы. Таким образом, нет способа спровоцировать неопределённое поведение, даже если вы передадите индекс за пределами длины среза. Другой пример — хеш-таблицы, которые хешируют предоставленный вами ключ за вас, вместо того чтобы позволять вам предоставить хеш самостоятельно; это гарантирует, что вы никогда не попытаетесь обратиться к ключу, используя неправильный хеш.

Однако в бесконечном стремлении к максимальной производительности некоторые разработчики могут обнаружить, что эти проверки безопасности добавляют чуть-чуть слишком много накладных расходов в их самых горячих циклах. Чтобы удовлетворить ситуации, где пиковая производительность имеет первостепенное значение и вызывающая сторона знает, что индексы находятся в границах, многие структуры данных предоставляют альтернативные версии определённых методов без проверок безопасности. Такие методы обычно включают в название слово `unchecked`, чтобы указать, что они слепо доверяют тому, что предоставленные аргументы безопасны, и не выполняют никаких из этих надоедливых, медленных проверок безопасности. Несколько примеров: `NonNull::new_unchecked`, `slice::get_unchecked`, `NonZero::new_unchecked`, `Arc::get_mut_unchecked` и `str::from_utf8_unchecked`.

На практике компромисс между безопасностью и производительностью для методов без проверок редко того стоит. Как всегда с оптимизацией производительности, сначала измеряйте, затем оптимизируйте.

Пользовательские инварианты (Custom Invariants)

Большинство случаев использования `unsafe` в той или иной степени опираются на пользовательские инварианты. То есть они опираются на инварианты, выходящие за рамки тех, что предоставляет сам Rust, и специфичные для конкретного приложения или библиотеки. Поскольку так много функций попадает в эту категорию, трудно дать хорошее общее резюме этого класса небезопасных функций. Вместо этого я приведу несколько примеров небезопасных функций с пользовательскими инвариантами, с которыми вы можете столкнуться на практике и захотеть использовать:

```
MaybeUninit::assume_init
```

Тип `MaybeUninit` — один из немногих способов, которыми вы можете хранить значения, которые не являются корректными для своего типа в Rust. Вы

можете думать о `MaybeUninit<T>` как о `T`, который в данный момент может быть недопустимо использовать как `T`. Например, `MaybeUninit<NonNull>` разрешено содержать нулевой указатель, `MaybeUninit<Box>` разрешено содержать висячий указатель на кучу, а `MaybeUninit<bool>` разрешено содержать битовый шаблон для числа 3 (обычно это должно быть 0 или 1). Это удобно, если вы конструируете значение бит за битом или имеете дело с обнулённой или неинициализированной памятью, которая в итоге будет сделана корректной (например, будучи заполненной через вызов `std::io::Read::read`). Функция `assume_init` утверждает, что `MaybeUninit` теперь содержит корректное значение для типа `T` и, следовательно, может быть использован как `T`.

`ManuallyDrop::drop`

Тип `ManuallyDrop` — это тип-обёртка вокруг типа `T`, который не сбрасывает этот `T`, когда сбрасывается `ManuallyDrop`. Или, иначе говоря, он отделяет сброс внешнего типа (`ManuallyDrop`) от сброса внутреннего типа (`T`). Он реализует безопасный доступ к `T` через `DerefMut<Target = T>`, но также предоставляет метод `drop` (отдельно от метода `drop` трейта `Drop`), чтобы сбросить обёрнутый `T`, *не* сбрасывая `ManuallyDrop`. То есть функция `drop` принимает `&mut self`, несмотря на то что сбрасывает `T`, и таким образом оставляет `ManuallyDrop` позади. Это удобно, если вам нужно явно сбросить значение, которое нельзя переместить, как, например, в реализациях трейта `Drop`. После сброса больше небезопасно пытаться обращаться к `T`, поэтому вызов `drop` небезопасен — он утверждает, что к `T` больше никогда не будут обращаться в дальнейшем.

`std::ptr::drop_in_place`

`drop_in_place` позволяет вам вызвать деструктор значения напрямую через указатель на это значение. Это небезопасно, потому что после вызова значение, на которое указывает указатель (`pointee`), останется на месте, так что если какой-то код затем попытается разыменовать указатель, ему придётся туго! Этот метод особенно полезен, когда вы можете захотеть повторно использовать память, как, например, в аренном аллокаторе (`arena allocator`), и вам нужно сбросить старое значение на месте, не освобождая окружающую память.

`Waker::from_raw`

В главе 9 мы говорили о типе `Waker` и о том, как он состоит из указателя на данные и `RawWaker`, который содержит вручную реализованную таблицу

виртуальных методов (vtable). После того как Waker сконструирован, сырые указатели на функции в vtable, такие как `wake` и `drop`, можно вызывать из безопасного кода (через `Waker::wake` и `drop(waker)` соответственно). `Waker::from_raw` — это место, где асинхронный исполнитель (executor) утверждает, что все указатели в его vtable на самом деле являются корректными указателями на функции, которые следуют контракту, изложенному в документации `RawWakerVTable`.

```
std::hint::unreachable_unchecked
```

Модуль `hint` содержит функции, которые дают компилятору подсказки об окружающем коде, но на самом деле не производят никакого машинного кода. Функция `unreachable_unchecked`, в частности, говорит компилятору, что для программы невозможно достичь некоторого участка кода во время выполнения. Это, в свою очередь, позволяет компилятору делать оптимизации на основе этого знания, например, устранять условные ветвления к этому месту. В отличие от макроса `unreachable!`, который паникует, если код всё же достигает рассматриваемой строки, последствия ошибочного `unreachable_unchecked` трудно предсказать. Оптимизации компилятора могут вызвать странное и трудноотлаживаемое поведение, не говоря уже о том, что ваша программа продолжит выполняться, когда то, что она считала истинным, на самом деле таковым не было!

```
std::ptr::{read,write}_{unaligned,volatile}
```

Модуль `ptr` содержит ряд функций, которые позволяют вам работать с *нестандартными* (*odd*) указателями — теми, которые не соответствуют предположениям, которые Rust обычно делает об указателях. Первые из этих функций — `read_unaligned` и `write_unaligned`, которые позволяют вам обращаться к указателям, указывающим на `T`, даже если этот `T` не хранится в соответствии с выравниванием (alignment) `T` (см. раздел о выравнивании в главе 3). Это может произойти, если `T` содержится непосредственно в байтовом массиве или иным образом упакован с другими значениями без надлежащего заполнения (padding). Вторая примечательная пара функций — `read_volatile` и `write_volatile`, которые позволяют вам оперировать указателями, не указывающими на обычную память. Конкретно, эти функции всегда будут обращаться к данному указателю (они не будут кешироваться в регистре, например, даже если вы прочитаете один и тот же указатель дважды подряд), и компилятор не будет переупорядочивать `volatile`-обращения относительно

других `volatile`-обращений. `Volatile`-операции удобны при работе с указателями, за которыми не стоит обычная память `DRAM`, — мы обсудим это подробнее в главе 13. В конечном счёте эти методы небезопасны, потому что они разыменовывают данный указатель (причём на принадлежащий `T`), так что вам как вызывающей стороне нужно подписаться под всеми контрактами, связанными с этим.

```
std::thread::Builder::spawn_unchecked
```

Обычный `thread::spawn`, который мы знаем и любим, требует, чтобы предоставленное замыкание было `'static`. Это ограничение проистекает из того факта, что порождённый поток может работать неопределённое количество времени; если бы нам было разрешено использовать ссылку, скажем, на стек вызывающей стороны, вызывающая сторона могла бы вернуться задолго до того, как порождённый поток завершится, делая ссылку недействительной. Однако иногда вы знаете, что некоторое не-`'static` значение в вызывающей стороне переживёт порождённый поток. Это может произойти, если вы снова присоединяете (`join`) поток до сброса рассматриваемого значения, или поток может перестать использовать значение после некоторой точки синхронизации с вызывающей стороной, и вызывающая сторона сбрасывает значение только после этой точки во времени. Здесь на помощь приходит `spawn_unchecked` — у него нет ограничения `'static`, и таким образом он позволяет вам реализовать эти случаи использования, пока вы готовы подписаться под контрактом, говорящим, что в результате не произойдёт никаких небезопасных обращений. Однако будьте осторожны с паниками: если вызывающая сторона паникует, она может сбросить значения раньше, чем вы планировали, и вызвать неопределённое поведение в порождённом потоке!

Обратите внимание, что все эти методы (и вообще все небезопасные методы в стандартной библиотеке) предоставляют явную документацию своих инвариантов безопасности, как и должно быть для любого небезопасного метода.

Реализация небезопасных трейтов (Implementing Unsafe Traits)

Небезопасные трейты небезопасны не для *использования*, а для *реализации*. Это потому, что небезопасному коду разрешено опираться на корректность (определённую документацией трейта) реализации небезопасных трейтов.

Например, чтобы реализовать небезопасный трейт `Send`, вам нужно написать `unsafe impl Send for ...`. Как и небезопасные функции, небезопасные трейты обычно имеют пользовательские инварианты, которые (или, по крайней мере, должны быть) прописаны в документации трейта. По этой причине трудно охватить небезопасные трейты как группу, так что здесь я приведу несколько распространённых примеров из стандартной библиотеки, которые стоит рассмотреть.

Send и Sync

Пара трейтов `Send` и `Sync` обозначает, что тип безопасно соответственно отправлять (`send`) или разделять (`share`) между границами потоков. Мы поговорим больше об этих трейтах в главе 11, но сейчас вам нужно знать, что это авто-трейты (`auto-traits`), и потому они обычно будут реализованы для большинства типов компилятором за вас. Но, как это обычно бывает с авто-трейтами, они не будут реализованы, если какой-либо из членов рассматриваемого типа сам не является `Send` или `Sync`.

В контексте `unsafe` это в первую очередь возникает из-за сырых указателей, которые не являются ни `Send`, ни `Sync`. На первый взгляд это может показаться разумным: у компилятора нет способа узнать, у кого ещё может быть сырой указатель на то же значение или как он может его использовать в данный момент, так что как может быть безопасно отправлять их между потоками? Но теперь, когда мы — закалённые небезопасные разработчики, этот аргумент кажется слабым: в конце концов, разыменование сырого указателя уже само по себе небезопасно, так разве обработка инвариантов `Send` и `Sync` не должна быть просто частью этого?

Строго говоря, сырые указатели могли бы быть и `Send`, и `Sync`. Проблема в том, что если бы они были такими, то типы, содержащие сырые указатели, автоматически были бы `Send` и `Sync` сами, даже если их автор мог этого не осознавать. Разработчик мог бы затем небезопасно разыменовывать сырые указатели, даже не задумываясь о том, что произошло бы, если бы эти типы были отправлены или разделены между границами потоков, и тем самым преднамеренно ввести неопределённое поведение. Вместо этого типы сырых указателей блокируют эти автоматические реализации как дополнительная защита, побуждая авторов небезопасного кода явно подписаться под контрактом о том, что они также соблюли инварианты `Send` и

Sync.

ПРИМЕЧАНИЕ Распространённая ошибка при небезопасных реализациях `Send` и `Sync` — забыть добавить ограничения для обобщённых параметров: `unsafe impl<T> Send for MyUnsafeType<T> {}`.

`GlobalAlloc`

Трейт `GlobalAlloc` — это то, как вы реализуете пользовательский аллокатор памяти в Rust. Мы не будем много говорить об этой теме в этой книге, но сам трейт интересен. Листинг 10-4 приводит обязательные методы трейта `GlobalAlloc`.

```
pub unsafe trait GlobalAlloc {  
    pub unsafe fn alloc(&self, layout: Layout) -> *mut u8;  
    pub unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout);  
}
```

Листинг 10-4: Трейт `GlobalAlloc` с его обязательными методами

По сути, у трейта есть один метод для выделения нового куска памяти, `alloc`, и один для освобождения куска памяти, `dealloc`. Аргумент `Layout` описывает размер и выравнивание типа, как мы обсуждали в главе 3. Каждый из этих методов небезопасен и несёт ряд инвариантов безопасности, которые должны соблюдать его вызывающие стороны.

`GlobalAlloc` также небезопасен, потому что он накладывает ограничения на реализующего этот трейт, а не на вызывающего его методы. Именно небезопасность трейта гарантирует, что реализующие соглашаются соблюдать инварианты, которые сам Rust предполагает у своего аллокатора памяти, как, например, в реализации `Box` в стандартной библиотеке. Если бы трейт не был небезопасным, реализующий мог бы безопасно реализовать `GlobalAlloc` способом, который производил бы невыровненные указатели или аллокации неправильного размера, и спровоцировать небезопасность в коде, который в иных обстоятельствах безопасен и предполагает, что аллокации корректны. Это нарушило бы правило о том, что безопасный код не должен иметь возможности спровоцировать небезопасность памяти в другом безопасном коде, и тем самым вызвать всевозможный хаос.

Удивительно не Unpin (Surprisingly Not Unpin)

Трейт `Unpin` не является небезопасным, что становится сюрпризом для многих разработчиков Rust. Это может оказаться сюрпризом даже после прочтения главы 9. В конце концов, трейт, как предполагается, должен гарантировать, что самоссылающиеся типы не станут недействительными, если их переместить после того, как они установили внутренние указатели (то есть после того, как их поместили в `Pin`). Кажется странным, что должно быть безопасно сделать так, чтобы тип можно было безопасно извлечь из `Pin`.

Есть две главные причины, почему `Unpin` не является небезопасным трейтом. Во-первых, в нём нет необходимости. Реализация `Unpin` для типа, который вы контролируете, не даёт вам возможности безопасно извлечь `!Unpin`-тип; для этого всё равно требуется небезопасность в виде вызова `Pin::new_unchecked` или `Pin::get_unchecked_mut`. Во-вторых, у вас уже есть безопасный способ извлечь из `Pin` любой тип, который вы контролируете: трейт `Drop`! Когда вы реализуете `Drop`, вам передаётся `&mut self`, даже если ваш тип ранее хранился в `Pin` и является `!Unpin`, и всё это без какой-либо небезопасности. Этот потенциал небезопасности покрывается инвариантами `Pin::new_unchecked`, которые должны быть соблюдены, чтобы вообще создать `Pin` такого `!Unpin`-типа.

Когда делать трейт небезопасным (When to Make a Trait Unsafe)

Немногие трейты в реальном мире небезопасны, но все те, что есть, следуют одному и тому же шаблону. Трейт должен быть небезопасным, если безопасный код, который предполагает, что трейт реализован правильно, может проявить небезопасность памяти, если трейт реализован *неправильно*.

Трейт `Send` — хороший пример, который стоит держать в уме: безопасный код может легко породить поток и передать значение этому порождённому потоку, но если бы `Rc` был `Send`, эта последовательность операций могла бы тривиально привести к небезопасности памяти. Подумайте, что произошло бы, если бы вы клонировали `Rc<Box>` и отправили его в другой поток: оба потока могли бы затем легко попытаться освободить `Box`, поскольку они не синхронизируют корректно доступ к счётчику ссылок `Rc`.

Трейт `Unpin` — хороший контрпример. Хотя возможно написать небезопасный код, который провоцирует небезопасность памяти, если `Unpin` реализован некорректно, никакой полностью безопасный код не может спровоцировать

небезопасность памяти из-за реализации `Unpin`. Не всегда легко определить, может ли трейт быть безопасным (действительно, трейт `Unpin` был небезопасным на протяжении большей части процесса RFC), но вы всегда можете ошибиться в сторону того, чтобы сделать трейт небезопасным, а затем сделать его безопасным позже, если поймёте, что это так! Просто держите в уме, что это обратно несовместимое изменение.

Также держите в уме, что лишь потому, что кажется, будто некорректная (или даже злонамеренная) реализация трейта вызвала бы много хаоса, это необязательно хорошая причина делать его небезопасным. Маркер `unsafe` должен в первую очередь использоваться для выделения случаев небезопасности памяти, а не чего-то, что может спровоцировать ошибки в бизнес-логике. Например, трейты `Eq`, `Ord`, `Deref` и `Hash` все безопасны, хотя в мире наверняка существует много кода, который пошёл бы вразнос, столкнувшись со злонамеренной реализацией, скажем, `Hash`, которая возвращала бы каждый раз другой случайный хеш при вызове. Это распространяется и на небезопасный код тоже — наверняка где-то существует небезопасный код, который был бы небезопасен по памяти при наличии такой реализации `Hash`, — но это не значит, что `Hash` должен быть небезопасным. То же верно для реализации `Deref`, которая каждый раз разыменовывала бы к другой (но корректной) цели. Такой небезопасный код опирался бы на контракт `Hash` или `Deref`, который на самом деле не выполняется; `Hash` никогда не утверждал, что он детерминирован, как и `Deref`. Или, точнее, авторы тех реализаций никогда не подписывались под тем, что это так, используя ключевое слово `unsafe`!

ПРИМЕЧАНИЕ Важное следствие того, что трейты вроде `Eq`, `Hash` и `Deref` безопасны, состоит в том, что небезопасный код может опираться только на *безопасность* безопасного кода, но не на его *корректность*. Это применимо не только к трейтам, но и ко всем взаимодействиям между небезопасным и безопасным кодом.

Большая ответственность (Great Responsibility)

Пока что мы в основном рассматривали различные вещи, которые вам разрешено делать с небезопасным кодом. Но небезопасному коду разрешено делать эти вещи только если он делает это безопасно. Даже хотя небезопасный код может, скажем, разыменовывать сырой указатель, он должен

делать это только если знает, что этот указатель корректен как ссылка на то, на что он указывает, в данный момент времени, с соблюдением всех обычных требований Rust к ссылкам. Иными словами, небезопасному коду даётся доступ к инструментам, которые можно использовать для совершения небезопасных вещей, но он должен совершать только безопасные вещи, используя эти инструменты.

Это, в свою очередь, поднимает вопрос о том, что вообще означает *безопасный* в первую очередь. Когда безопасно разыменовывать указатель? Когда безопасно делать `transmute` между двумя разными типами? В этом разделе мы изучим некоторые из ключевых инвариантов, которые нужно держать в уме при использовании силы `unsafe`, рассмотрим некоторые распространённые подводные камни и познакомимся с некоторыми инструментами, которые помогают писать более безопасный небезопасный код.

Точные правила, касающиеся того, что значит для кода Rust быть безопасным, всё ещё прорабатываются. На момент написания книги рабочая группа `Unsafe Code Guidelines Working Group` усердно трудится над окончательным определением всех «можно» и «нельзя», но многие вопросы остаются нерешёнными. Большая часть советов в этом разделе более или менее устоялась, но там, где это не так, я обязательно укажу на это. Если ничего другого, я надеюсь, что этот раздел научит вас быть осторожными при выдвижении предположений, когда вы пишете небезопасный код, и побудит вас перепроверять справочник по Rust (`Rust reference`), прежде чем вы объявите свой код готовым к продакшену.

Что может пойти не так? (What Can Go Wrong?)

Мы не можем по-настоящему вникнуть в правила, которым должен следовать небезопасный код, не поговорив о том, что происходит, если вы нарушаете эти правила. Допустим, вы изменяемо обращаетесь к значению из нескольких потоков одновременно, конструируете невыровненную ссылку или разыменовываете висячий указатель — что тогда?

Небезопасный код, который не является в конечном счёте безопасным, называется имеющим *неопределённое поведение* (*undefined behavior*). Неопределённое поведение обычно проявляется одним из трёх способов: вообще никак, через видимые ошибки или через невидимое повреждение.

Первый — счастливый случай: вы написали код, который по-настоящему небезопасен, но компилятор сгенерировал нормальный код, который компьютер, на котором вы запускаете код, выполняет нормальным образом. К сожалению, это счастье очень хрупкое. Стоит появиться новой и чуть более умной версии компилятора, или какому-то окружающему коду заставить компилятор применить другую оптимизацию, и код может перестать делать что-то нормальное и скатиться в один из худших случаев. Даже если тот же код компилируется тем же компилятором, если он запускается на другой платформе или хостит программу, она может вести себя иначе! Вот почему важно избегать неопределённого поведения, даже если в данный момент всё, кажется, работает нормально. Не делать этого — всё равно что играть во второй раунд русской рулетки только потому, что вы пережили первый.

Видимые ошибки легче всего отловить как неопределённое поведение. Если вы разыменовываете нулевой указатель, например, ваша программа (по всей вероятности) аварийно завершится с ошибкой, которую вы затем сможете отладить до первопричины. Сама эта отладка может оказаться сложной, но по крайней мере у вас есть уведомление, что что-то не так. Видимые ошибки могут также проявляться менее серьёзными способами, такими как взаимоблокировки (deadlocks), искажённый вывод или паники, которые печатаются, но не вызывают завершения программы, — все они говорят вам, что в вашем коде есть ошибка, которую вам нужно исправить.

Худшее проявление неопределённого поведения — когда нет немедленного видимого эффекта, но состояние программы невидимо повреждено. Суммы транзакций могут немного отличаться от того, какими они должны быть, резервные копии могут быть незаметно повреждены, или случайные биты внутренней памяти могут быть выставлены наружу внешним клиентам. Неопределённое поведение может вызвать продолжающееся повреждение или крайне редкие сбои. Часть проблемы с неопределённым поведением в том, что, как следует из названия, поведение не-безопасного небезопасного кода не определено — компилятор может полностью устранить его, кардинально изменить семантику кода или даже неправильно скомпилировать окружающий код. Что это сделает с вашей программой, полностью зависит от того, что делает рассматриваемый код. Непредсказуемое воздействие неопределённого поведения — причина, по которой *всё* неопределённое поведение следует считать серьёзной ошибкой, независимо от того, как оно *в данный момент* проявляется.

ПОЧЕМУ НЕОПРЕДЕЛЁННОЕ ПОВЕДЕНИЕ? (WHY UNDEFINED BEHAVIOR?) Аргумент, который часто всплывает в разговорах о неопределённом поведении, состоит в том, что компилятор должен выдавать ошибку, если код проявляет неопределённое поведение, вместо того чтобы делать что-то странное и непредсказуемое. Тогда было бы почти невозможно написать плохой небезопасный код!

К сожалению, это было бы невозможно, потому что неопределённое поведение редко бывает явным или очевидным. Вместо этого обычно происходит то, что компилятор просто применяет оптимизации в предположении, что код следует спецификации. Если это окажется не так — что редко бывает ясно вплоть до времени выполнения, — трудно предсказать, каков может быть эффект этого. Может быть, оптимизация всё ещё корректна, и ничего плохого не произойдёт; но может быть, и нет, и семантика кода в итоге будет немного отличаться от неоптимизированной версии.

Если бы мы сказали разработчикам компиляторов, что им не разрешено делать никаких предположений о лежащем в основе коде, то на самом деле мы бы говорили им, что они не могут выполнять широкий спектр оптимизаций, которые они сегодня реализуют с большим успехом. Почти все сложные оптимизации делают предположения о том, что рассматриваемый код может и не может делать согласно спецификации языка.

Если вам нужна хорошая иллюстрация того, как спецификации и оптимизации компилятора странным образом взаимодействуют так, что трудно назначить вину, я рекомендую прочитать пост в блоге Ральфа Юнга (Ralf Jung) «We Need Better Language Specs» (<https://www.ralfj.de/blog/2020/12/14/provenance.html>).

Корректность (Validity)

Пожалуй, самая важная концепция, которую нужно понять перед написанием небезопасного кода, — это *корректность (validity)*, которая диктует правила относительно того, какие значения населяют данный тип, — или, менее формально, правила для значений типа. Концепция проще, чем звучит, так что давайте углубимся в несколько конкретных примеров.

Ссылочные типы (Reference Types)

Rust очень строг относительно того, какие значения могут содержать его ссылочные типы. В частности, ссылки никогда не должны «висеть», всегда должны быть выровнены и всегда должны указывать на корректное значение для своего целевого типа. Кроме того, разделяемая и эксклюзивная ссылка на данную область памяти никогда не могут существовать одновременно, и не может существовать нескольких эксклюзивных ссылок на одну область памяти. Эти правила применяются независимо от того, использует ли ваш код ссылки или нет — вам не разрешено создавать нулевую ссылку, даже если вы затем немедленно её отбросите!

У разделяемых ссылок есть дополнительное ограничение: значение, на которое указывают (pointee), не должно меняться в течение времени жизни ссылки. То есть любое значение, которое содержит pointee, должно оставаться точно таким же на протяжении своего времени жизни. Это применяется транзитивно, так что если у вас есть `&` на тип, содержащий `*mut T`, вам не разрешено когда-либо изменять `T` через этот `*mut`, даже хотя вы могли бы написать код для этого, используя `unsafe`. Единственное исключение из этого правила — значение, обёрнутое типом `UnsafeCell`. Все остальные типы, которые предоставляют внутреннюю изменяемость (interior mutability), такие как `Cell`, `RefCell` и `Mutex`, внутри используют `UnsafeCell`.

Интересный результат строгих правил Rust для ссылок состоит в том, что многие годы было невозможно безопасно взять ссылку на поле упакованной (packed) или частично неинициализированной структуры, использующей `repr(Rust)`. Поскольку `repr(Rust)` оставляет компоновку (layout) типа неопределённой, единственным способом получить адрес поля было написать `&some_struct.field as *const _`. Однако если `some_struct` упакована, то `some_struct.field` может быть не выровнено, и таким образом создание `&` на него недопустимо! Более того, если `some_struct` не полностью инициализирована, то сама ссылка `some_struct` не может существовать! В Rust 1.51.0 был стабилизирован макрос `ptr::addr_of!`, который добавил механизм для прямого получения ссылки на поле, не создавая сначала ссылку, решив эту конкретную проблему. Внутри он реализован с использованием того, что называется *сырыми ссылками* (raw references) (не путать с сырыми указателями), которые напрямую создают указатели на свои операнды, а не идут через ссылку. Сырые ссылки были введены в RFC 2582, но ещё не стабилизированы на момент написания книги.

Примитивные типы (Primitive Types)

Некоторые из примитивных типов Rust имеют ограничения на то, какие значения они могут содержать. Например, `bool` определён как имеющий размер 1 байт, но ему разрешено содержать только значение `0x00` или `0x01`, а `char` не разрешено содержать суррогат (surrogate) или значение выше `char::MAX`. Большинство примитивных типов Rust, да и вообще большинство типов Rust в целом, также нельзя сконструировать из неинициализированной памяти. Эти ограничения могут показаться произвольными, но опять-таки часто проистекают из необходимости включить оптимизации, которые иначе были бы невозможны.

Хорошая иллюстрация этого — нишевая оптимизация, которую мы кратко обсуждали ранее в этой главе, говоря о типах указателей. Напомним, что нишевая оптимизация прячет дискриминант перечисления (enum discriminant) в обёрнутый тип в определённых случаях. Например, поскольку ссылка никогда не может быть полностью нулевой, `Option<&T>` может использовать все нули для представления `None` и таким образом избежать траты дополнительного байта (плюс заполнение) на хранение байта дискриминанта. Компилятор может оптимизировать булевы значения тем же способом и потенциально пойти ещё дальше. Рассмотрим `Option<Option<bool>>`. Поскольку компилятор знает, что `bool` — это либо `0x00`, либо `0x01`, он волен использовать `0x02` для представления `Some(None)` и `0x03` для представления `None`. Очень мило и аккуратно! Но если бы кто-то пришёл и обработал байт `0x03` как `bool`, а затем поместил это значение в `Option<Option<bool>>`, оптимизированный таким способом, произошли бы плохие вещи.

Стоит повторить, что неважно, реализует ли компилятор Rust в настоящее время эту оптимизацию или нет. Дело в том, что ему это разрешено, и потому любой небезопасный код, который вы пишете, должен соответствовать этому контракту или рисковать наткнуться на ошибку позже, если поведение изменится.

Типы владеющих указателей (Owned Pointer Types)

Типы, которые указывают на память, которой владеют, такие как `Box` и `Vec`, обычно подвержены тем же оптимизациям, как если бы они держали эксклюзивную ссылку на память, на которую указывают, если только к ним не обращаются явно через разделяемую ссылку. В частности, компилятор

предполагает, что память, на которую указывают, не разделяется и не имеет других псевдонимов (aliased) где-либо ещё, и делает оптимизации на основе этого предположения. Например, если бы вы извлекли указатель из `Box`, а затем сконструировали два `Box` из этого же указателя и обернули их в `ManuallyDrop`, чтобы предотвратить двойное освобождение, вы, вероятно, вступили бы на территорию неопределённого поведения. Это так, даже если вы когда-либо обращаетесь к внутреннему типу только через разделяемые ссылки. (Я говорю «вероятно», потому что это ещё не полностью устоялось в справочнике по языку, но возник примерный консенсус.)

Хранение недопустимых значений (Storing Invalid Values)

Иногда вам нужно сохранить значение, которое в данный момент недопустимо для своего типа. Самый распространённый пример этого — если вы хотите выделить кусок памяти для некоторого типа `T`, а затем прочитать в него байты, скажем, из сети. Пока все байты не считаны, память не будет корректным `T`. Даже если вы просто попытаетесь прочитать байты в срез из `u8`, вам сначала пришлось бы обнулить эти `u8`, потому что конструирование `u8` из неинициализированной памяти — тоже неопределённое поведение.

Тип `MaybeUninit<T>` — это механизм Rust для работы со значениями, которые не являются корректными. `MaybeUninit<T>` хранит ровно `T` (он `#[repr(transparent)]`), но компилятор знает, что не нужно делать никаких предположений о корректности этого `T`. Он не будет предполагать, что ссылки не нулевые, что `Box<T>` не висячий или что `bool` — это либо 0, либо 1. Это значит, что безопасно содержать `T`, за которым стоит неинициализированная память, внутри `MaybeUninit` (как следует из названия). `MaybeUninit` также очень полезный инструмент в другом небезопасном коде, где вам нужно временно сохранить значение, которое может быть недопустимым. Может быть, вам нужно сохранить `Box<T>` с псевдонимом или припрятать суррогат `char` на секунду — `MaybeUninit` вам друг.

Есть всего три вещи, которые вы обычно будете делать с `MaybeUninit`: создавать его с помощью метода `MaybeUninit::uninit`, писать в его содержимое с помощью `MaybeUninit::as_mut_ptr` или снова брать внутренний `T`, когда он становится корректным, с помощью `MaybeUninit::assume_init`. Как следует из названия, `uninit` создаёт новый `MaybeUninit` того же размера, что и `T`, который изначально содержит неинициализированную память. Метод `as_mut_ptr` даёт вам сырой

указатель на внутренний `T`, в который вы затем можете писать; ничто не мешает вам читать из него, но чтение из любого неинициализированного бита — неопределённое поведение. И наконец, небезопасный метод `assume_init` потребляет `MaybeUninit<T>` и возвращает его содержимое как `T` после утверждения, что лежащая в основе память теперь составляет корректный `T`.

Листинг 10-5 показывает пример того, как мы могли бы использовать `MaybeUninit` для безопасной инициализации байтового массива без явного обнуления.

```
fn fill(gen: impl FnMut() -> Option<u8>) {
    let mut buf = [MaybeUninit::<u8>::uninit(); 4096];
    let mut last = 0;
    for (i, g) in std::iter::from_fn(gen).take(4096).enumerate() {
        buf[i] = MaybeUninit::new(g);
        last = i + 1;
    }
    // Безопасность: все u8 вплоть до last инициализированы.
    let init: &[u8] = unsafe { MaybeUninit::slice_assume_init_ref(&buf[..
last]) };
    // ... сделать что-нибудь с init ...
}
```

Листинг 10-5: Использование `MaybeUninit` для безопасной инициализации массива

Если бы мы вместо этого объявили `buf` как `[0; 4096]`, это потребовало бы, чтобы функция сначала записала все эти нули на стек перед выполнением, даже если она собирается снова перезаписать их все вскоре после. Обычно это не оказало бы заметного влияния на производительность, но если бы это было в достаточно горячем цикле, могло бы! Здесь мы вместо этого позволяем массиву хранить любые значения, которые случайно оказались на стеке на момент вызова функции, и затем перезаписываем только то, что нам в итоге нужно.

ПРИМЕЧАНИЕ Будьте осторожны со сбросом частично инициализированной памяти. Если паника вызывает неожиданный ранний сброс до того, как `MaybeUninit<T>` был полностью инициализирован, вам придётся позаботиться о том, чтобы сбросить только те части `T`, которые теперь корректны, если таковые есть. Вы *можете* просто сбросить `MaybeUninit` и оставить лежащую в основе память забытой, но если она содержит, скажем, `Vox`, у вас может

Паники (Panics)

Важный и часто упускаемый из виду аспект обеспечения безопасности кода, использующего небезопасные операции, состоит в том, что код также должен быть готов обрабатывать паники. В частности, как мы кратко обсуждали в главе 6, обработчик паник Rust по умолчанию на большинстве платформ не аварийно завершит вашу программу при панике, а вместо этого *разматывает* (*unwind*) текущий поток. Разматывающая паника, по сути, сбрасывает всё в текущей области видимости, возвращается из текущей функции, сбрасывает всё в области видимости, охватывающей функцию, и так далее вниз по стеку, пока не достигнет первого стекового фрейма для текущего потока. Если вы не учтёте разматывание в своём небезопасном коде, у вас могут быть проблемы. Например, рассмотрите код в листинге 10-6, который пытается эффективно затолкнуть много значений в `Vec` за раз.

```
impl<T: Default> Vec<T> {
    pub fn fill_default(&mut self) {
        let fill = self.capacity() - self.len();
        if fill == 0 { return; }
        let start = self.len();
        unsafe {
            self.set_len(start + n);
            for i in 0..fill { *self.get_unchecked_mut(start + i) =
T::default(); }
        }
    }
}
```

Листинг 10-6: На первый взгляд безопасный метод для заполнения вектора значениями `Default`

Подумайте, что происходит с этим кодом, если вызов `T::default` паникует. Сначала `fill_default` сбросит все свои локальные значения (которые являются просто целыми числами), а затем вернётся. Вызывающая сторона затем сделает то же самое. В какой-то момент вверх по стеку мы добираемся до владельца `Vec`. Когда владелец сбрасывает вектор, у нас возникает проблема: длина вектора теперь указывает, что мы владеем большим количеством `T`, чем мы фактически произвели из-за вызова `set_len`. Например, если самый первый вызов `T::default` запаниковал, когда мы намеревались заполнить восемь элементов, это значит, что `Vec::drop` вызовет `drop` для восьми `T`, которые на

самом деле содержат неинициализированную память!

Исправление в этом случае простое: код должен обновлять длину *после* записи всех элементов. Мы бы не осознали, что есть проблема, если бы не рассмотрели внимательно эффект разматывающих паник на корректность нашего небезопасного кода.

Когда вы прочёсываете свой код в поисках такого рода проблем, вам стоит высматривать любые инструкции, которые могут запаниковать, и обдумывать, безопасен ли ваш код, если они паникуют. Как вариант, проверьте, можете ли вы убедить себя, что рассматриваемая инструкция никогда не запаникует. Обращайте особое внимание на всё, что вызывает предоставленный пользователем код, — в таких случаях у вас нет контроля над паниками, и вам следует предполагать, что пользовательский код запаникует.

Похожая ситуация возникает, когда вы используете оператор `return` для раннего возврата из функции. Если вы делаете это, убедитесь, что ваш код всё ещё безопасен, если он не выполняет оставшуюся часть кода в функции. Это реже застаёт врасплох с `return`, поскольку вы выбрали его явно, но это стоит держать в поле зрения.

Проверка сброса (The Drop Check)

Проверщик заимствований (borrow checker) Rust, по сути, является сложным инструментом для обеспечения корректности (soundness) кода во время компиляции, что, в свою очередь, даёт Rust способ выражать «безопасность» кода. Как именно проверщик заимствований выполняет свою работу, выходит за рамки этой книги, но одна проверка — *проверка сброса (drop check)* — стоит того, чтобы рассмотреть её подробно, поскольку она имеет некоторые прямые последствия для небезопасного кода. Чтобы понять проверку сброса, давайте на секунду поставим себя на место компилятора Rust и рассмотрим два фрагмента кода. Сначала взгляните на маленький трёхстрочник в листинге 10-7, который берёт изменяемую ссылку на переменную, а затем сразу же мутирует эту же переменную.

```
let mut x = true;
let foo = Foo(&mut x);
x = false;
```

Листинг 10-7: Реализация Foo диктует, должен ли этот код компилироваться

Не зная определения `Foo`, можете ли вы сказать, должен ли этот код компилироваться или нет? Когда мы устанавливаем `x = false`, всё ещё существует `foo`, который будет сброшен в конце области видимости. Мы знаем, что `foo` содержит изменяемое заимствование `x`, что указывало бы, что изменяемое заимствование, необходимое для модификации `x`, недопустимо. Но в чём вред разрешить это? Оказывается, мутация `x` проблематична только если `Foo` реализует `Drop` — если `Foo` не реализует `Drop`, то мы знаем, что `Foo` не будет трогать ссылку на `x` после её последнего использования. Поскольку это последнее использование происходит до того, как нам понадобится эксклюзивная ссылка для присваивания, мы можем разрешить этот код! С другой стороны, если `Foo` реализует `Drop`, мы не можем разрешить этот код, поскольку реализация `Drop` может использовать ссылку на `x`.

Теперь, когда вы разогрелись, взгляните на листинг 10-8. В этом не столь прямолинейном фрагменте кода изменяемая ссылка зарыта ещё глубже.

```
fn barify<'a>(_: &'a mut i32) -> Bar<Foo<'a>> { .. }
let mut x = true;
let foo = barify(&mut x);
x = false;
```

Листинг 10-8: Реализации и Foo, и Bar диктуют, должен ли этот код компилироваться

Опять же, не зная определений `Foo` и `Bar`, можете ли вы сказать, должен ли этот код компилироваться или нет? Давайте рассмотрим, что происходит, если `Foo` реализует `Drop`, а `Bar` — нет, поскольку это самый интересный случай. Обычно, когда `Bar` выходит из области видимости или иным образом сбрасывается, ему всё равно придётся сбросить `Foo`, что, в свою очередь, означает, что код должен быть отклонён по той же причине, что и раньше: `Foo::drop` мог бы обратиться к ссылке на `x`. Однако `Bar` может вообще не содержать `Foo` напрямую, а вместо этого хранить `PhantomData<Foo<'a>>` или `&'static Foo<'a>`, в каком случае код на самом деле в порядке — даже хотя `Bar` сбрасывается, `Foo::drop` никогда не вызывается, и к ссылке на `x` никогда не обращаются. Это тот вид кода, который мы хотим, чтобы компилятор принимал, потому что человек сможет определить, что он в порядке, даже если компилятор сочтёт затруднительным это обнаружить.

Логика, которую мы только что прошли, и есть проверка сброса. Обычно она не слишком сильно влияет на небезопасный код, поскольку её поведение по умолчанию соответствует ожиданиям пользователей, за одним крупным исключением: висячие обобщённые параметры (*dangling generic parameters*). Представьте, что вы реализуете свой собственный тип `Box<T>`, и кто-то помещает `&mut x` в него, как мы делали в листинге 10-7. Вашему типу `Box` нужно реализовать `Drop`, чтобы освобождать память, но он не обращается к `T` помимо его сброса. Поскольку сброс `&mut` ничего не делает, должно быть совершенно нормально для кода снова обратиться к `&mut x` после последнего раза, когда `Box` использовался, но до того, как он сброшен! Чтобы поддерживать такие типы, в Rust есть нестабильная возможность под названием `dropck_eyepatch` (потому что она делает проверку сброса частично слепой). Эта возможность, вероятно, останется нестабильной навсегда и предназначена служить лишь временным аварийным выходом, пока не будет разработан надлежащий механизм. Возможность `dropck_eyepatch` добавляет атрибут `#[may_dangle]`, который вы можете добавить как префикс к обобщённым временам жизни и типам в реализации `Drop` типа, чтобы сказать механизму проверки сброса, что вы не будете использовать аннотированное время жизни или тип помимо их сброса. Вы используете это, написав:

```
unsafe impl<#[may_dangle] T> Drop for ..
```

Этот аварийный выход позволяет типу объявить, что данный обобщённый параметр не используется в `Drop`, что включает случаи использования вроде `Box<&mut T>`. Однако он также вводит новую проблему, если ваш `Box<T>` держит сырой указатель на кучу `*mut T` и позволяет `T` висеть с помощью `#[may_dangle]`. В частности, `*mut T` заставляет проверку сброса Rust думать, что ваш `Box<T>` не владеет `T`, и потому он не вызывает `T::drop`. В сочетании с утверждением `may_dangle` о том, что мы не обращаемся к `T`, когда `Box<T>` сбрасывается, проверка сброса теперь заключает, что нормально, если `T` не живёт до тех пор, пока `Box` сбрасывается (как наш укороченный `&mut x` в листинге 10-8). Но это неправда, поскольку мы *действительно* вызываем `T::drop`, который сам может обратиться, скажем, к ссылке на упомянутый `x`.

К счастью, исправление простое: мы добавляем `PhantomData<T>`, чтобы сказать проверке сброса, что хотя `Box<T>` не держит никакого `T` и не обращается к `T` при сбросе, он всё же владеет `T` и сбросит один, когда `Box` сбрасывается. Листинг 10-9 показывает, как выглядел бы наш гипотетический тип `Box`.


```
struct Box<T> {
    t: NonNull<T>, // NonNull, а не *mut, для ковариантности (глава 2)
    _owned: PhantomData<T>, // чтобы проверка сброса осознала, что мы сбрасываем T
}
unsafe impl<#[may_dangle] T> for Box<T> { /* ... */ }
```

Листинг 10-9: Определение Box, максимально гибкое с точки зрения проверки сброса

Это взаимодействие тонкое и его легко упустить, но оно возникает только когда вы используете нестабильный атрибут `#[may_dangle]`. Надеюсь, этот подраздел послужит предупреждением, что когда вы увидите `unsafe impl Drop` в дикой природе в будущем, вы будете знать, что нужно высматривать ещё и `PhantomData<T>!`

ПРИМЕЧАНИЕ Ещё одно соображение для небезопасного кода, касающееся `Drop`, состоит в том, чтобы убедиться, что если у вас есть `Type<T>`, который позволяет `T` продолжать жить после того, как `self` сброшен, вам также нужно добавить `T: 'static`. Например, если вы реализуете отложенную сборку мусора (delayed garbage collection), вам нужно также добавить `T: 'static`. Иначе, если `T = WriteOnDrop<&mut U>`, последующее обращение или сброс `T` могли бы спровоцировать неопределённое поведение!

Приведение типов (Casting)

Как мы обсуждали в главе 3, два разных типа, которые оба являются `#[repr(Rust)]`, могут быть представлены в памяти по-разному, даже если у них есть поля одного типа и в том же порядке. Это, в свою очередь, означает, что не всегда очевидно, безопасно ли приводить между двумя разными типами. На самом деле Rust даже не гарантирует, что два экземпляра одного типа с обобщёнными аргументами, которые сами скомпонованы одинаково, представлены одинаково. Например, в листинге 10-10 не гарантируется, что `A` и `B` имеют одинаковое представление в памяти.

```
struct Foo<T> {
    one: bool,
    two: PhantomData<T>,
}
struct Bar;
struct Baz;
```

```
type A = Foo<Bar>;
type B = Foo<Baz>;
```

Листинг 10-10: Компоновка типов непредсказуема.

Отсутствие гарантий для `repr(Rust)` важно держать в уме, когда вы делаете приведение типов в небезопасном коде, — лишь потому, что два типа ощущаются так, будто они должны быть взаимозаменяемы, это необязательно так. Приведение между двумя типами, которые имеют разные представления, — быстрый путь к неопределённому поведению. На момент написания книги сообщество Rust активно прорабатывает точные правила того, как типы представляются, но пока даётся очень мало гарантий, так что это то, с чем нам приходится работать.

Даже если бы было гарантировано, что идентичные типы имеют одинаковое представление в памяти, вы всё равно столкнулись бы с той же проблемой при вложении типов. Например, хотя `UnsafeCell<T>`, `MaybeUninit<T>` и `T` все на самом деле просто держат `T`, и вы можете приводить их друг к другу сколько душе угодно, всё это вылетает в окно, как только у вас есть, например, `Option<MaybeUninit<T>>`. Хотя `Option<T>` может быть в состоянии воспользоваться нишевой оптимизацией (используя какое-то недопустимое значение `T` для представления `None` у `Option`), `MaybeUninit<T>` может содержать любой битовый шаблон, так что эта оптимизация не применяется, и приходится держать дополнительный байт для дискриминанта `Option`.

Не только оптимизации могут вызвать расхождение компоновок, когда в игру вступают типы-обёртки. В качестве примера возьмите код в листинге 10-11; здесь компоновка `Wrapper<PhantomData<u8>>` и `Wrapper<PhantomData<i8>>` совершенно разная, хотя предоставленные типы оба пусты!

```
struct Wrapper<T: SneakyTrait> {
    item: T::Sneaky,
    iter: PhantomData<T>,
}
trait SneakyTrait {
    type Sneaky;
}
impl SneakyTrait for PhantomData<u8> {
    type Sneaky = ();
}
impl SneakyTrait for PhantomData<i8> {
    type Sneaky = [u8; 1024];
}
```

Листинг 10-11: Типы-обёртки затрудняют правильное приведение типов.

Всё это не означает, что вы никогда не сможете приводить типы в Rust. Всё становится намного проще, например, когда вы контролируете все вовлечённые типы и их реализации трейтов, или если типы являются `#[repr(C)]`. Вам просто нужно осознавать, что Rust даёт очень мало гарантий о представлениях в памяти, и писать код соответственно!

Справляемся со страхом (Coping with Fear)

С этой главой, в основном оставшейся позади, вы теперь можете бояться небезопасного кода больше, чем до того, как начали. Хотя это понятно, важно подчеркнуть, что не только *возможно* писать безопасный небезопасный код, но и большую часть времени это даже несложно. Ключ в том, чтобы убедиться, что вы обращаетесь с небезопасным кодом аккуратно; это половина дела. Будьте по-настоящему уверены, что нет безопасной реализации, которую вы могли бы использовать вместо этого, прежде чем прибегать к `unsafe`.

В оставшейся части этой главы мы рассмотрим некоторые техники и инструменты, которые могут помочь вам быть более уверенным в корректности вашего небезопасного кода, когда нет способа его обойти.

Управляйте границами небезопасности (Manage Unsafe Boundaries)

Соблазнительно рассуждать о небезопасности *локально*; то есть рассматривать, безопасен ли код в небезопасном блоке, который вы только что написали, не задумываясь слишком сильно о его взаимодействии с остальной частью кодовой базы. К сожалению, такого рода локальные рассуждения часто возвращаются, чтобы укусить вас. Хороший пример этого — трейт `Unpin`: вы можете написать какой-то код для своего типа, который использует `Pin::new_unchecked` для создания закреплённой ссылки на поле типа, и этот код может быть полностью безопасным, когда вы его пишете. Но затем в какой-то более поздний момент вы (или кто-то ещё) можете добавить безопасную реализацию `Unpin` для указанного типа, и внезапно небезопасный код перестаёт быть безопасным, хотя он и близко не находится к новой

реализации `impl!`

Безопасность — это свойство, которое можно проверить только на *границе приватности* (*privacy boundary*) всего кода, относящегося к небезопасному блоку. *Граница приватности* здесь — не столько формальный термин, сколько попытка описать «любую часть вашего кода, которая может повозиться с небезопасными битами». Например, если вы объявляете публичный тип `Foo` в модуле `bar`, помеченном `pub` или `pub(crate)`, то любой другой код в том же крейте может реализовывать методы и трейты для `Foo`. Так что если безопасность вашего небезопасного кода зависит от того, что `Foo` не реализует определённые трейты или методы с определёнными сигнатурами, вам нужно помнить о том, чтобы перепроверять безопасность этого небезопасного блока всякий раз, когда вы добавляете `impl` для `Foo`. Если же, с другой стороны, `Foo` не виден всему крейту, то намного меньший набор областей видимости способен добавлять проблемные реализации, и таким образом риск случайно добавить реализацию, которая нарушает инварианты безопасности, соответственно снижается. Если `Foo` приватен, то только текущий модуль и любые подмодули могут добавлять такие реализации.

То же правило применяется к доступу к полям: если безопасность небезопасного блока зависит от определённых инвариантов над полями типа, то любой код, который может трогать эти поля (включая безопасный код), попадает в границу доверия небезопасного блока. И здесь тоже минимизация границы приватности — лучший подход: код, который не может добраться до полей, не может испортить ваши инварианты!

Поскольку небезопасный код часто требует таких далеко идущих рассуждений, лучшая практика — инкапсулировать небезопасность в вашем коде настолько хорошо, насколько можете. Предоставьте небезопасность в форме единственного модуля и стремитесь дать этому модулю интерфейс, который полностью безопасен. Таким образом вам нужно будет проверять внутренности этого модуля только на ваши инварианты. Или, ещё лучше, поместите небезопасные биты в их собственный крейт, чтобы вы не могли оставить никаких дыр открытыми случайно!

Однако не всегда возможно полностью инкапсулировать сложные небезопасные взаимодействия в единственный безопасный интерфейс. Когда это так, постарайтесь сузить части публичного интерфейса, которые должны быть небезопасными, так чтобы у вас было лишь очень небольшое их число,

дайте им названия, которые ясно сообщают, что нужна осторожность, и затем строго их документируйте.

Иногда возникает соблазн убрать маркер `unsafe` с внутренних API, чтобы не приходилось расставлять `unsafe {}` по всему коду. В конце концов, внутри вашего кода вы знаете, что никогда не будете вызывать `frobnify`, если ранее вызывали `bazzify`, верно? Удаление аннотации `unsafe` может привести к более чистому коду, но обычно это плохое решение в долгосрочной перспективе. Через год, когда ваша кодовая база разрастётся, вы выгрузите из головы некоторые из инвариантов безопасности и «просто захотите быстро слепить эту одну фичу», и есть шанс, что вы непреднамеренно нарушите один из этих инвариантов. А поскольку вам не нужно набирать `unsafe`, вы даже не подумаете проверить. Плюс, даже если вы сами никогда не делаете ошибок, как насчёт других контрибьюторов в ваш код? В конечном счёте более чистый код — недостаточно хороший аргумент, чтобы убирать намеренно «шумный» маркер `unsafe`.

Читайте и пишите документацию (Read and Write Documentation)

Само собой разумеется, что если вы пишете небезопасную функцию, вы должны задокументировать условия, при которых эту функцию безопасно вызывать. Здесь важны и ясность, и полнота. Не упускайте никаких инвариантов, даже если вы их уже где-то ещё прописали. Если у вас есть тип или модуль, который требует определённых глобальных инвариантов — инвариантов, которые должны всегда соблюдаться для всех использований типа, — то напомните читателю, что они также должны соблюдать глобальные инварианты и в документации каждой небезопасной функции. Предполагайте, что разработчики читают документацию по принципу «по мере необходимости» и, вероятно, не читали вашу тщательно написанную документацию уровня модуля, и им нужно дать подсказку об этом.

Что может быть менее очевидно, так это то, что вам также следует документировать все небезопасные реализации и блоки — думайте об этом как о предоставлении доказательства того, что вы действительно соблюдаете контракт, который требует рассматриваемая операция. Например, `slice::get_unchecked` требует, чтобы предоставленный индекс находился в границах среза; когда вы вызываете этот метод, поместите комментарий прямо над ним, объясняющий, откуда вы знаете, что индекс действительно

гарантированно в границах. В некоторых случаях инварианты небезопасного блока обширны, и ваши комментарии могут оказаться длинными. Это хорошо. Я много раз ловил ошибки, пытаюсь написать комментарий безопасности для небезопасного блока и осознавая на полпути, что я на самом деле не соблюдаю ключевой инвариант. Вы также поблагодарите себя через год, когда вам нужно будет модифицировать этот код и убедиться, что он всё ещё безопасен. И так же поступит контрибьютор в ваш проект, который только что наткнулся на этот небезопасный вызов и хочет понять, что происходит.

Прежде чем вы слишком глубоко погрузитесь в написание небезопасного кода, я также настоятельно рекомендую прочитать Rustonomicon (<https://doc.rust-lang.org/nomicon/>) от корки до корки. Есть так много деталей, которые легко упустить и которые регулярно будут возвращаться, чтобы укусить вас, если вы о них не знаете. Мы охватили многие из них в этой главе, но никогда не помешает знать больше. Вам также следует активно использовать справочник по Rust всякий раз, когда вы в сомнениях. Он постоянно дополняется, и есть шанс, что если вы даже немного не уверены, верно ли какое-то ваше предположение, справочник укажет на это. Если же нет, рассмотрите возможность открыть issue, чтобы это было добавлено!

Проверяйте свою работу (Check Your Work)

Хорошо, итак, вы написали какой-то небезопасный код, дважды и трижды проверили все инварианты и думаете, что он готов к запуску. Прежде чем вы выпустите его в продакшен, есть некоторые автоматизированные инструменты, через которые вам следует прогнать свой набор тестов (у вас же есть набор тестов, верно?).

Первый из них — Miri, интерпретатор представления среднего уровня (Mid-level Intermediate Representation Interpreter). Miri не компилирует ваш код в машинный код, а вместо этого интерпретирует код Rust напрямую. Это даёт Miri гораздо больше видимости того, что делает ваша программа, что, в свою очередь, позволяет ему проверять, что ваша программа не делает ничего откровенно плохого, например, не читает из неинициализированной памяти. Miri может отлавливать множество очень тонких и специфичных для Rust ошибок и является спасением для любого, кто пишет небезопасный код.

К сожалению, поскольку ему приходится интерпретировать код, чтобы его выполнить, код, запущенный под Miri, часто работает на порядки медленнее

своего скомпилированного аналога. По этой причине его следует использовать действительно только для выполнения вашего набора тестов. Он также может проверить только тот код, который фактически выполняется, и потому не отловит проблемы в путях кода, которых ваш набор тестов не достигает. Вам стоит думать о Miri как о расширении вашего набора тестов, а не его замене.

Существуют также инструменты, известные как *санитайзеры (sanitizers)*, которые инструментируют машинный код для обнаружения ошибочного поведения во время выполнения. Накладные расходы и точность этих инструментов сильно различаются, но один особенно широко любимый инструмент — AddressSanitizer от Google. Он обнаруживает большое количество ошибок памяти, таких как использование после освобождения (use-after-free), переполнения буфера и утечки памяти, все из которых являются распространёнными симптомами некорректного небезопасного кода. В отличие от Miri, эти инструменты работают с машинным кодом и потому, как правило, довольно быстры — обычно в пределах того же порядка величины. Но, как и Miri, они ограничены анализом кода, который фактически выполняется, так что здесь тоже жизненно важен солидный набор тестов.

Ключ к эффективному использованию этих инструментов — автоматизировать их через ваш конвейер непрерывной интеграции (continuous integration), чтобы они запускались для каждого изменения, и гарантировать, что вы добавляете регрессионные тесты по мере обнаружения ошибок. Инструменты лучше отлавливают проблемы по мере улучшения качества вашего набора тестов, так что, добавляя новые тесты по мере исправления известных ошибок, вы, так сказать, зарабатываете двойные очки!

Наконец, не забывайте щедро рассыпать утверждения (assertions) по всему небезопасному коду. Паника всегда лучше, чем провоцирование неопределённого поведения! Проверяйте все свои предположения утверждениями, какие можете, — даже такие вещи, как размер `usize`, если вы полагаетесь на это ради безопасности. Если вы беспокоитесь о стоимости во время выполнения, используйте макросы `debug_assert*` и конструкцию `if cfg!(debug_assertions) || cfg!(test)`, чтобы выполнять их только в отладочном и тестовом контекстах.

КАРТОЧНЫЙ ДОМИК? (A HOUSE OF CARDS?) Небезопасный код может нарушить все гарантии безопасности Rust, и это часто

преподносится как причина, по которой аргумент о безопасности всего Rust — фарс. Беспокойство состоит в том, что достаточно лишь одного бита некорректного небезопасного кода, чтобы весь дом обрушился и вся безопасность была потеряна. Сторонники этого аргумента затем иногда утверждают, что по меньшей мере только небезопасный код должен иметь возможность вызывать небезопасный код, так чтобы небезопасность была видна вплоть до самого высокого уровня приложения.

Аргумент понятен — это правда, что безопасность кода Rust опирается на безопасность всего того небезопасного кода, который он в итоге вызывает. И действительно, если часть этого небезопасного кода некорректна, это может иметь последствия для безопасности программы в целом. Однако этот аргумент упускает то, что *все* успешные безопасные языки предоставляют средство для языковых расширений, которые не выразимы на (безопасном) поверхностном языке, обычно в форме кода, написанного на C или ассемблере. Точно так же, как Rust опирается на корректность своего небезопасного кода, безопасность этих языков опирается на корректность этих расширений.

Rust отличается тем, что у него нет отдельного языка расширений, а вместо этого он написан на том, что равнозначно диалекту Rust (небезопасный Rust). Это позволяет гораздо более тесную интеграцию между безопасным и небезопасным кодом, что, в свою очередь, снижает вероятность ошибок из-за рассогласования (impedance mismatch) на интерфейсе между ними или из-за того, что разработчики знакомы с одним, но не с другим. Более тесная интеграция также облегчает написание инструментов, которые анализируют корректность взаимодействия небезопасного кода с безопасным, как иллюстрируют инструменты вроде Miri. Поскольку небезопасный Rust продолжает подчиняться проверщику заимствований для любой операции, которая не является явно небезопасной, остаётся много проверок безопасности на местах, которых нет, когда разработчикам приходится спускаться к языку вроде C.

Итоги (Summary)

В этой главе мы прошлись по силам, которые даёт ключевое слово `unsafe`, и по ответственности, которую мы принимаем, используя эти силы. Мы также поговорили о последствиях написания небезопасного небезопасного кода и о том, как вам действительно следует думать об `unsafe` как о способе поклясться компилятору, что вы вручную проверили, что указанный код всё же безопасен. В следующей главе мы погрузимся в параллелизм (concurrency) в Rust и посмотрим, как заставить все те ядра в вашем новом блестящем компьютере тянуть в одном направлении!